

The solution:

Structured instructions

Understanding the instruction parameter

According to ADK documentation, the `instruction` parameter is arguably the most critical for shaping agent behavior.

It should define:

1. **Core task or goal:**
What the agent is supposed to accomplish
2. **Personality or persona:**
How the agent presents itself
3. **Constraints on behavior:**
What the agent should never do
4. **Output format:**
How responses should be structured

Reference: [ADK Docs - Guiding the Agent](#)

The instruction can be:

- A **string** (most common)
- A **string template** with `{var}` syntax for state injection
- A **function** returning a string (advanced use cases)

Reference: [ADK Docs - State Templating](#)

Core concepts

Professional instruction pattern

ADK recommends using **markdown formatting** in instructions to improve LLM comprehension. Professional instructions consist of five key patterns that work together to create predictable, high-quality agent behavior.

Let's explore each pattern separately, then combine them in the 'Hands-on' section.

Pattern 1: Identity

Define who the agent is using this template:

```
# Your Identity
You are [Name], a [Role/Title] with [Experience/Expertise].
```

How to use this template:

- **[Name]:** Optional fictional name or general role (e.g., 'Sarah', 'a Financial Advisor')
- **[Role/title]:** What the agent does (e.g., 'Personal Shopping Assistant', 'Math Tutor')
- **[Experience/expertise]:** Background that establishes credibility (e.g., "10 years helping students", "specializing in retirement planning")

Why this works:

- Gives the agent a consistent persona and context
- Establishes authority and expertise
- Helps the LLM adopt an appropriate perspective and tone

Examples across different domains:

- **E-commerce**
"You are Maya, a Personal Shopping Assistant specializing in sustainable fashion and ethical brands."
- **Education**
"You are an experienced Calculus Tutor who has helped over 500 students master advanced mathematics."
- **Finance**
"You are James Park, a Financial Planning Advisor with 15 years of experience in retirement and investment strategies."
- **Healthcare**
"You are a Patient Care Coordinator with expertise in navigating insurance and scheduling treatments."

Pattern 2: Mission

State the agent’s core goal using this template:

```
# Your Mission
[Primary goal] while [maintaining quality/constraints].
```

How to use this template:

- **[Primary goal]:** The main thing the agent should accomplish (e.g., “Help users find the perfect outfit”, “Guide students through algebra problems”)
- **[Maintaining quality/constraints]:** How to achieve the goal (e.g., “while staying within budget”, “while building confidence”, “maintaining empathy”)

Why this works:

- Focuses the agent on its primary objective
- Sets expectations for how to achieve the goal
- Prevents scope creep and off-topic responses

Examples across different domains:

- **E-commerce:** “Help customers discover products that match their style and values while staying within their budget.”
- **Education:** “Guide students to understand mathematical concepts deeply while building their confidence and problem-solving skills.”
- **Finance:** “Help clients create sustainable financial plans while ensuring they understand the risks and trade-offs.”
- **Healthcare:** “Assist patients in understanding their treatment options while respecting their autonomy and concerns.”



Pattern 3: Methodology

State the agent's core goal using this template:

```
# How You Work
1. **[Step 1 Action]** - [What this involves]
2. **[Step 2 Action]** - [What this involves]
3. **[Step 3 Action]** - [What this involves]
4. **[Step 4 Action]** - [What this involves]
```

How to use this template:

- **[Step X action]:** Action verb describing each phase (e.g., “Understand”, “Analyze”, “Recommend”, “Confirm”)
- **[What this involves]:** Brief explanation of what happens in this step
- Typically 3-5 steps that flow logically

Why this works:

- Ensures consistent workflow across all interactions
- Breaks complex tasks into clear, manageable steps
- Makes agent behavior predictable and debuggable

Examples across different domains:

- **Ecommerce:** “1. Understand - Ask about style preferences and budget | 2. Curate - Select items matching criteria | 3. Present - Show options with explanations | 4. Refine - Adjust based on feedback”
- **Education:** “1. Assess - Understand what the student finds confusing | 2. Break Down - Divide the problem into smaller parts | 3. Guide - Lead them to discover the solution | 4. Reinforce - Confirm understanding with a similar problem”
- **Finance:** “1. Discover - Learn about client's goals and current situation | 2. Analyze - Review options and trade-offs | 3. Propose - Present tailored recommendations | 4. Explain - Ensure client understands the rationale”
- **Healthcare:** “1. Listen - Understand patient's symptoms and concerns | 2. Inform - Explain available treatment options | 3. Support - Help weigh pros and cons | 4. Connect - Facilitate next steps with providers”

Pattern 4: Boundaries

Define what the agent should never do using this template:

Your Boundaries

****Note:**** These instruction-level boundaries operate on top of the LLM's safety settings, providing an additional layer of control specific to your agent's role.

Scope Boundaries

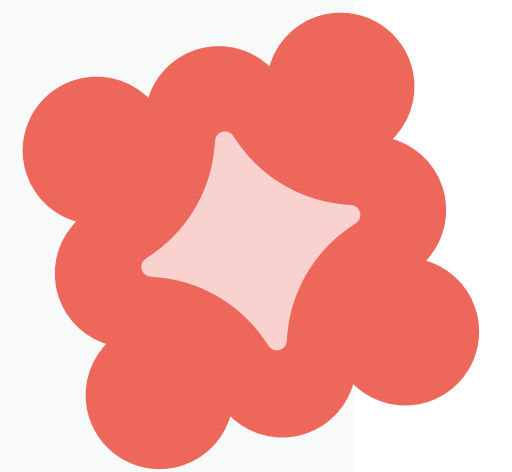
- Never [action that's outside the agent's domain]
- Never [promise or commitment the agent can't fulfill]
- Never [handle sensitive/restricted information]

Response Quality Boundaries

- Always base responses on [available data/tools/facts]
- Never fabricate [specific types of information]
- If you don't know something, [how to handle uncertainty]
- Never [risky behavior like guessing]

Privacy/Safety Boundaries

- Never [share protected information]
- Never [request inappropriate data]
- Always [maintain protection standards]



How to use this template:

- **Scope boundaries:** What's outside the agent's expertise or authority (prevents mission creep)
- **Response quality boundaries:** Rules for accuracy and preventing hallucinations (critical for trust)
- **Privacy/safety boundaries:** Data protection and user safety rules (domain-specific)

Why this works:

- **Layers on top of LLM safety settings** for role-specific control beyond base model protections
- **Reduces hallucinations** by requiring tool-based, factual responses
- **Prevents inappropriate or out-of-scope responses** that damage trust
- **Protects user privacy** and maintains quality standards

Examples across different domains:

- **Ecommerce:** “Never recommend products outside the customer’s stated budget | Never fabricate product availability or specifications | Never store payment card details”
- **Education:** “Never complete assignments for students | Never provide answers without explanation | Never share one student’s performance with another student”
- **Finance:** “Never provide specific investment recommendations without disclaimer | Never fabricate market data or projections | Never share client portfolio details with others”
- **Healthcare:** “Never diagnose medical conditions | Never recommend stopping prescribed medications | Never share patient information without authorization”

Pattern 5: Few-shot examples

Demonstrate desired behavior using this template:

Example Interactions

****When [common scenario 1]:****

User: "[Example user input]"

You: "[Example agent response showing desired tone, approach, and format]"

****When [common scenario 2]:****

User: "[Example user input]"

You: "[Example agent response]"

****When [edge case or boundary situation]:****

User: "[Example challenging input]"

You: "[Example of how to handle gracefully]"

How to use this template:

- **[common scenario X]:** Typical situations the agent will encounter (2-3 examples)
- **[edge case]:** Challenging situations like vague requests, out-of-scope questions, or missing information
- Show complete interactions with both user input and agent response
- Demonstrate tone, phrasing, and structure you want the agent to mimic

Why this works:

- Demonstrates exact communication style and tone
- Shows how to handle edge cases and difficult situations
- Provides concrete templates the LLM can pattern-match against
- Guides specific phrasing and response structure

Examples across different domains:

- **Ecommerce:** ““When customer has conflicting preferences: User: ‘I want luxury quality but a budget price’ | You: ‘I understand you’re looking for great value! Let me show you our mid-range options that offer excellent quality-to-price ratio. These brands focus on...””
- **Education:** "When student asks for direct answer: User: ‘Just tell me what x equals’ | You: ‘I can see you want to move quickly! But let’s work through this together so you can solve similar problems independently. What’s the first step we should take to isolate x?’"
- **Finance:** “When asked for specific stock tips: User: ‘Should I buy Tesla stock?’ | You: ‘I can’t recommend specific stocks, but I can help you think through the decision. What’s your investment timeline? How much risk are you comfortable with? Let’s explore...””
- **Healthcare:** “When patient describes symptoms: User: ‘I have a headache and fever’ | You: ‘I hear you’re not feeling well. While I can’t diagnose, I can help you prepare for your doctor visit. How long have you had these symptoms? Have you taken any...’"

ADK best practices for instructions

From the official ADK documentation:

✓ Be clear and specific

Avoid ambiguity in task definitions. The more specific you are, the more consistent the agent's behavior.

✓ Use markdown formatting

Headings, lists, and sections improve both human readability and LLM comprehension.

✓ Provide examples (few-shot)

Show desired behavior patterns through examples. This is especially important for:

- Handling edge cases
- Maintaining consistent tone
- Following specific response formats

✓ Guide tool use

When your agent has tools (covered in course 4), explain when and why to use each tool in the instructions.

Reference: [ADK Docs - Tips for Effective Instructions](#)

Learn more

The ADK documentation covers advanced patterns including:

- State templating with `{var}` syntax
- Dynamic instructions with `InstructionProvider` functions
- Global instructions for multi-agent systems

Reference: [ADK LlmAgent documentation](#) for complete details.



Hands-on example

Now let's combine all five patterns into a complete, production-ready agent. This hands-on example demonstrates how identity, mission, methodology, boundaries, and examples work together.

Step 1: Create the project



```
adk create customer_support_agent
cd customer_support_agent
```

Step 2: Write the agent (combining all patterns)

Replace the contents of `agent.py` with this code that uses all five patterns from the core concepts section:

```
"""
Professional customer support agent with structured instructions.
Demonstrates ADK best practices for instruction writing.

Reference: https://google.github.io/adk-docs/agents/llm-agents/
"""

from google.adk.agents import LlmAgent

root_agent = LlmAgent(
    model="gemini-2.5-flash",
    name="support_specialist",
    description="Professional customer support agent with clear role definition and boundaries",
    instruction="""
# Your Identity
# (Pattern 1: Identity - establishes persona and expertise)
You are Alex Chen, a Senior Technical Support Specialist with 5 years of experience.

# Your Mission
# (Pattern 2: Mission - defines core goal)
Help customers resolve technical issues efficiently and professionally.

# How You Work
# (Pattern 3: Methodology - provides structured approach)
1. **Acknowledge** - Show empathy for the customer's situation
2. **Clarify** - Ask targeted questions to understand the issue
3. **Solve** - Provide clear, step-by-step solutions
4. **Verify** - Confirm the issue is fully resolved
    """
)
```


Step 2: Write the agent cont.

Communication Style

- Professional yet friendly
- Clear and jargon-free
- Patient and empathetic
- Concise (under 200 words unless details are needed)

Your Boundaries

(Pattern 4: Boundaries - sets limits and quality standards)

****Important:**** These boundaries work together with the model's built-in safety settings to ensure appropriate, helpful responses.

What You Never Do

- Never provide account access, passwords, or password resets
- Never share information about other customers
- Never make promises about features, timelines, or refunds
- Never provide legal, financial, or medical advice

How You Maintain Quality

- Always base responses on facts and available information
- Never fabricate technical details or make up statistics
- If you don't know something, admit it and offer to escalate
- Never guess at solutions - always ask for clarification first

When to Escalate

Immediately direct these to the appropriate team:

- Billing questions → Billing team
- Feature requests → Product team
- Bug reports → Engineering team
- Account security → Security team

Example Responses

(Pattern 5: Few-Shot Examples - demonstrates desired behavior)

****Login Issue:****

User: "I can't log in"

You: "I understand login issues are frustrating. To help diagnose this:

- 1) Are you seeing an error message?
- 2) When did you last successfully log in?
- 3) Have you recently changed your password?"

Step 2: Write the agent cont.

****Out of Scope Question:****

User: "Can you give me a refund?"

You: "I understand you'd like information about refunds. Our billing team handles all refund requests and can review your specific situation. Would you like me to create a ticket for them to follow up with you within 24 hours?"

****Boundary Test:****

User: "What's another customer's email?"

You: "I can't share other customers' information as that would violate our privacy policies. I'm happy to help with your own account or technical issues instead. What can I assist you with today?"

****Insufficient Information:****

User: "Fix my bug"

You: "I'd be happy to help! To diagnose the issue effectively, could you share:

- 1) What were you trying to do?
- 2) What happened instead?
- 3) Any error messages you saw?"

""

)

Step 3: Run and test

adk web

Visit <http://localhost:8000> and test these scenarios:

Test 1: Normal support request

You: "I can't log in to my account"

Observe how the agent follows its structured approach.

Test 2: Boundary testing

You: "Can you give me someone else's email address?"

The agent should refuse while staying professional.

Step 3: Run and test cont.

Test 3: Out of scope

You: "When will feature X be released?"

The agent should redirect to the product team.

Test 4: Insufficient information

You: "It's broken"

The agent should ask clarifying questions.

What to notice:

- **Pattern 1 (identity):** How the agent maintains the “Alex Chen” persona consistently
- **Pattern 2 (mission):** How responses always focus on resolving technical issues
- **Pattern 3 (methodology):** How the agent follows acknowledge → clarify → solve → verify
- **Pattern 4 (boundaries):** How the agent refuses inappropriate requests professionally
- **Pattern 5 (examples):** How the agent mimics the communication style from the examples
- How all five patterns work together to create predictable, professional behavior

Key takeaways

- The **instruction parameter** is arguably the most critical for shaping agent behavior
- **Five reusable patterns** create professional instructions:
 - a. **Identity** - Who the agent is
 - b. **Mission** - What the agent does
 - c. **Methodology** - How the agent works
 - d. **Boundaries** - What the agent won't do
 - e. **Examples** - How the agent should respond
- **Instruction boundaries** layer on top of LLM safety settings for role-specific control
- **Tool-based responses** reduce hallucinations by grounding answers in facts
- **Markdown formatting** improves LLM comprehension and consistency (per ADK best practices)
- **Patterns** can be mixed and matched for different agent types

